Distributed Payment Gateway

Project Roadmap & Progress Tracker

7	24	4–6	∞
Phases	Weeks	Months	Things learned

Guiding principle: Depth beats breadth. An interviewer at Razorpay or Stripe will ask *why* you made every decision. One phase built exceptionally well is worth more than seven phases built shallowly. Never rush — understand before you move on.

What tier-1 companies look for in a backend project

Signal	What it means in practice
You understand the why	Not just 'I used Kafka' — but why Kafka and not Redis Pub/Sub for this use case
You handled failure	Happy path is easy. Show retry logic, circuit breakers, dead letter queues
You designed for scale	Stateless services, horizontal scaling, no shared in-memory state
Clean, readable code	Naming, separation of concerns, no god classes — reviewers read your code

Phase 1 — Core Foundation — Two Services Talking — ■ Weeks 1–3

Goal

Get a single payment flowing end-to-end across two separate Spring Boot applications. No async, no Kafka, no complexity — just two services, one HTTP call, one database each. This forces you to feel the pain of distributed systems immediately.

What you are building

- **Payment Service (port 8080)** — what you already built today, production-hardened
- **Mock Bank Service (port 8081)** — new, simulates a real bank

Mock Bank Service design

- Single endpoint: POST /bank/process
- Accepts: amount, paymentId, paymentMethod
- Configurable success rate via application.yaml — e.g. 80% approve, 20% decline
- Configurable simulated latency — random 100–500ms delay to feel real

- Returns: APPROVED or DECLINED with a reason code

Updated payment flow

Client → POST /payments → Payment Service creates PENDING payment → calls Mock Bank → Bank returns APPROVED/DECLINED → Payment Service updates status to SUCCESS/FAILED → returns final response to client

Key design rule

Note: Payment Service owns the payments table. Mock Bank Service owns nothing — it is stateless. Neither service ever touches the other's database. This is the core rule of microservices.

Concepts you will learn

RestClient / WebClient	Inter-service HTTP	Service boundaries	Timeout handling	Stateless design

What the interviewer will ask

- What happens if Mock Bank takes 30 seconds to respond?
- How do you set a timeout on the HTTP call?
- What status is the payment in if the bank call times out?

Phase 1 checklist

■	**Mock Bank Service created**	Separate Spring Boot app on port 8081
■	**POST /bank/process endpoint**	Returns APPROVED or DECLINED
■	**Configurable success rate**	Set in application.yaml, e.g. bank.successRate=0.8
■	**Simulated latency**	Random sleep 100–500ms
■	**Payment Service calls Mock Bank**	Using RestClient with timeout configured
■	**Status updated after bank call**	PENDING → SUCCESS or FAILED
■	**Timeout handled gracefully**	If bank call times out, payment stays FAILED with reason
■	**Test full flow in Bruno**	Create payment, verify status is SUCCESS or FAILED

Phase 2 — Async Processing — Kafka

■ Weeks 4–6

Goal

Replace the synchronous HTTP call between Payment Service and Mock Bank with an asynchronous event-driven flow using Kafka. This is the single most important architectural step in the whole project — it's what makes the system truly distributed.

Why Kafka and not the HTTP call you already have

Problem with synchronous HTTP: If Mock Bank is slow, Payment Service is blocked. If Mock Bank is down, the payment fails immediately. Under high load, slow banks cause thread exhaustion in Payment Service. Kafka decouples them — Payment Service fires an event and moves on. Mock Bank processes when it can. Both services survive each other's failures.

The new async flow

Client → POST /payments → Payment Service saves PENDING → publishes PaymentInitiated event to Kafka → returns 202 Accepted immediately Mock Bank Service consumes PaymentInitiated → processes → publishes PaymentProcessed event Payment Service consumes PaymentProcessed → updates status to SUCCESS/FAILED

Kafka topics to create

- **payment.initiated** — Payment Service publishes, Mock Bank consumes
- **payment.processed** — Mock Bank publishes, Payment Service consumes

The hardest part — idempotent consumers

Kafka guarantees at-least-once delivery. That means the same event can arrive twice. Your consumers must handle duplicates without creating duplicate payments or double-updating status. Your idempotency key from Phase 1 is your weapon here — check it before processing any event.

Concepts you will learn

Apache Kafka	Producers & consumers	Consumer groups	At-least-once delivery	Idempotent consumers	Offset management

Mini project before starting Phase 2

Note: Build a standalone Kafka mini project first — produce and consume messages, understand offsets, consumer groups, and what happens when a consumer crashes mid-processing. Don't skip this.

Phase 2 checklist

■	**Kafka running locally**	Via Docker or local install
■	**payment.initiated topic created**	Payment Service publishes on payment creation
■	**payment.processed topic created**	Mock Bank publishes result
■	**Mock Bank is now a Kafka consumer**	Consumes PaymentInitiated, processes, publishes result
■	**Payment Service consumes result**	Updates payment status from PaymentProcessed event
■	**POST /payments returns 202**	Accepted — not the final status, processing is async
■	**GET /payments/{id} shows status**	Client polls this to get final SUCCESS/FAILED
■	**Idempotent consumer implemented**	Duplicate events do not create duplicate payments
■	**Test consumer crash recovery**	Kill consumer mid-processing, restart, verify it recovers

Phase 3 — Merchant Side — Webhooks

■ Weeks 7–9

Goal

Add the merchant dimension. Real payment gateways serve merchants — businesses that integrate with your gateway to accept payments. Merchants register, get API keys, and receive webhook notifications when payment status changes.

What you are building

- **Merchant Service** — merchants register, receive an API key, configure a webhook URL
- **Webhook Service** — notifies merchants via HTTP POST when payment status changes
- **API Key auth** — every payment request must include a valid merchant API key

Webhook flow

Payment status changes to SUCCESS/FAILED → Webhook Service publishes WebhookTriggered event → Webhook worker POSTs to merchant's callback URL → If merchant server returns 2xx: mark delivered → If merchant server returns non-2xx or times out: retry with exponential backoff

Retry strategy for webhooks

- Attempt 1: immediately
- Attempt 2: after 30 seconds
- Attempt 3: after 5 minutes
- Attempt 4: after 30 minutes
- Attempt 5: after 2 hours — if still failing, mark as DEAD and alert

The Outbox Pattern — do not skip this

Note: When payment status changes and you publish a webhook event, these are two operations — DB update and Kafka publish. If the DB commits but Kafka publish fails, the merchant never gets notified. The Outbox Pattern solves this: write the event to an outbox table in the same DB transaction as the status update, then a separate process publishes from the outbox. This guarantees at-least-once webhook delivery.

Concepts you will learn

Webhook patterns	Exponential backoff	API key auth	Outbox pattern	Retry queues	Dead letter handling

Phase 3 checklist

■	**Merchant Service created**	POST /merchants/register returns API key
■	**API key validation**	Every payment request validated against merchant
■	**Webhook URL stored per merchant**	Merchants configure their callback URL on registration
■	**Webhook Service created**	Listens for payment status changes
■	**Webhook delivery implemented**	HTTP POST to merchant callback URL
■	**Retry with exponential backoff**	5 attempts with increasing delays
■	**Outbox pattern implemented**	Webhook events written in same DB transaction
■	**Dead webhook handling**	Mark as DEAD after max retries, log for investigation
■	**Test webhook delivery**	Use webhook.site or a local echo server to verify

Phase 4 — Multiple Payment Methods

■ Weeks 10–12

Goal

Support Card, UPI, and NetBanking as distinct payment methods, each routed to its own Mock Bank processor with different behaviour. This is where design patterns become real — the Strategy Pattern is not an academic exercise here.

The Strategy Pattern in practice

Define a PaymentProcessor interface with a process(PaymentRequest) method. Implement CardProcessor, UpiProcessor, NetBankingProcessor — each with different simulated latency and behaviour. A PaymentRouter picks the right processor based on the paymentMethod field in the request. Adding a new payment method = new class, zero changes elsewhere.

Different behaviour per method

Method	Simulated Latency	Success Rate	Failure Modes
Card	200–500ms	85%	Insufficient funds, card expired, CVV mismatch
UPI	50–150ms	92%	UPI ID not found, daily limit exceeded
NetBanking	1000–3000ms	78%	Bank timeout, session expired

Concepts you will learn

Strategy pattern	Payment routing	Polymorphism at scale	Failure mode simulation	Extensible design

Phase 4 checklist

■	**PaymentProcessor interface defined**	Single process() method
■	**CardProcessor implemented**	200–500ms latency, 85% success
■	**UpiProcessor implemented**	50–150ms latency, 92% success
■	**NetBankingProcessor implemented**	1–3s latency, 78% success
■	**PaymentRouter implemented**	Picks correct processor from paymentMethod field
■	**PaymentRequest updated**	Includes paymentMethod enum field
■	**Failure reason codes returned**	Specific codes per method, not generic FAILED
■	**Test all three methods**	Verify different latencies and failure modes in Bruno

Phase 5 — Resilience — Circuit Breakers & Retry

■ Weeks 13–16

Goal

Make the system fault-tolerant. A payment gateway that crashes when a downstream service misbehaves is not production-ready. Resilience4j gives you circuit breakers, retry, timeout, and rate limiter as annotations — simple to add, profound in effect.

The four resilience patterns

Pattern	What it does	When to use it
Circuit Breaker	After N failures, stop calling the service for T seconds	Mock Bank keeps timing out
Retry	On transient failure, retry N times before giving up	Network blip, temporary unavailability
Timeout	Fail fast if call takes longer than T milliseconds	Slow bank dragging your threads
Rate Limiter	Limit calls per second to protect downstream services	Prevent overloading Mock Bank

Dead Letter Queue — the safety net

When a Kafka message fails processing after all retries, it goes to a Dead Letter Topic (DLT). Every consumer in your system needs a DLT. Build a simple DLT monitor endpoint that shows how many messages are stuck and why — this is what on-call engineers look at at 3am.

Concepts you will learn

Resilience4j	Circuit breaker states	Exponential backoff	Bulkhead pattern	Dead letter topics	Fault tolerance

Phase 5 checklist

■	**Resilience4j dependency added**	spring-cloud-starter-circuitbreaker-resilience4j
■	**Circuit breaker on bank calls**	@CircuitBreaker — open after 5 failures in 10s
■	**Retry on transient failures**	@Retry — 3 attempts with 500ms between
■	**Timeout configured**	@TimeLimiter — fail if bank takes more than 3s
■	**Dead Letter Topic per consumer**	Failed messages routed to .DLT topic
■	**DLT monitor endpoint**	GET /admin/dlt-messages shows stuck messages
■	**Test circuit breaker**	Kill Mock Bank, verify circuit opens after N failures
■	**Test recovery**	Restart Mock Bank, verify circuit closes again

Phase 6 — Settlement Service ■ Weeks 17–20

Goal

Build the end-of-day batch job that aggregates successful payments per merchant and generates settlement reports. This is how merchants actually receive their money in real payment systems. Spring Batch is the right tool — built for exactly this job.

What settlement means

All day: payments are processed and marked SUCCESS in your DB. End of day (e.g. 11:59 PM): Settlement Service runs a batch job — finds all SUCCESS payments not yet settled, groups by merchant, sums amounts, generates a settlement record, marks payments as SETTLED. In a real system this triggers an actual bank transfer. In yours, it generates a report.

Spring Batch concepts you will use

- **Job** — the entire settlement run
- **Step** — read unsettled payments, process, write settlement records
- **ItemReader** — reads SUCCESS payments from DB in chunks of 100
- **ItemProcessor** — groups by merchant, sums amounts
- **ItemWriter** — writes settlement records, marks payments SETTLED
- **@Scheduled** — triggers the job daily at 11:59 PM

New DB table needed

```
settlements: id, merchant_id, total_amount, payment_count, period_date, status, created_at
```

Concepts you will learn

Spring Batch	Chunk processing	Scheduled jobs	Batch idempotency	Settlement patterns	Financial reconciliation

Phase 6 checklist

■	**Settlement Service created**	Separate Spring Boot app
■	**Settlements table created**	Via Flyway migration
■	**Spring Batch job configured**	Reader → Processor → Writer
■	**Chunk size set to 100**	Process 100 payments at a time, not all at once
■	**@Scheduled trigger**	Runs daily at 11:59 PM via cron expression
■	**Idempotent job runs**	Running twice on same day produces same result
■	**Settlement report endpoint**	GET /settlements?merchantId=X&date;=Y
■	**Payments marked SETTLED**	Status updated after successful settlement
■	**Test manual trigger**	POST /admin/settlements/trigger for testing

Phase 7 — Polish — What Gets You Shortlisted

■ Weeks 21–24

Goal

The code is done. Now make it impossible to ignore on a resume. This phase is about presentation, observability, and documentation — the things that separate a learning project from something that looks production-grade.

API Gateway — single entry point

Add Spring Cloud Gateway as the single entry point for all client traffic. It handles routing to the right service, rate limiting per API key, and authentication. Suddenly your system looks like a real gateway — one URL, all services behind it.

Distributed Tracing — follow a payment across services

Add Micrometer + Zipkin. Every request gets a trace ID that flows through all services. You can see exactly how long each service took for a given payment. This is what interviewers mean by observability — and most candidates don't have it.

Metrics Dashboard — Prometheus + Grafana

Expose Spring Boot Actuator metrics, scrape with Prometheus, visualise in Grafana. Build a dashboard showing: payments per second, success rate, p99 latency, circuit breaker state, Kafka consumer lag. Screenshot this for your resume.

The README — most underrated part of the project

Write your README as if you are handing it to a senior engineer at Razorpay. It should contain: (1) Architecture diagram with all services labeled. (2) Tech stack and why each technology was chosen. (3) Key design decisions — why SAGA, why Kafka, why Redis for idempotency. (4) How to run the system locally. (5) API documentation with example requests. (6) What you would do differently with more time. The last point especially — it shows self-awareness and engineering maturity.

Phase 7 checklist

■	**Spring Cloud Gateway added**	Single entry point, routes to all services
■	**Rate limiting on API key**	100 requests/minute per merchant
■	**Micrometer + Zipkin tracing**	Trace ID flows through all services
■	**Prometheus metrics exposed**	Via Spring Boot Actuator
■	**Grafana dashboard built**	Payments/sec, success rate, latency, circuit state
■	**Swagger on all services**	springdoc-openapi, clean API docs
■	**Architecture diagram in README**	All services, databases, queues labeled
■	**Design decisions documented**	Why each technology choice was made
■	**Bruno collection committed**	All endpoints with example requests in the repo
■	**Grafana dashboard screenshot**	Add to README — it is visual proof of observability

Mini projects to build before each phase

You already built the Redis + PostgreSQL mini project today. Each phase below requires a focused mini project first — so you are never learning a new technology and applying it to complex business logic at the same time.

Before Phase	Mini Project	What to build	Est. Time
Phase 1	RestClient mini project	Two Spring Boot apps, one calls the other via HTTP, handle timeouts	2–3 days
Phase 2	Kafka mini project	Produce messages, consume, simulate crash mid-processing, recover	4–5 days
Phase 3	Webhook mini project	HTTP POST to webhook.site, retry on failure, exponential backoff	2–3 days
Phase 5	Resilience4j mini project	Circuit breaker on a flaky endpoint, observe state transitions	2–3 days
Phase 6	Spring Batch mini project	Read CSV → process → write to DB, understand chunks and steps	3–4 days
Phase 7	Docker mini project	Containerise one service, run Postgres and Redis via compose	3–5 days

Full tech stack

Category	Technology	Used in
Language	Java 21	All services
Framework	Spring Boot 4	All services
Database	PostgreSQL	Payment, Merchant, Settlement services
Cache	Redis (Memurai on Windows)	Payment Service — idempotency
Messaging	Apache Kafka	All async communication from Phase 2
Resilience	Resilience4j	Circuit breaker, retry, timeout — Phase 5
Batch	Spring Batch	Settlement Service — Phase 6
API Gateway	Spring Cloud Gateway	Phase 7 — single entry point
Tracing	Micrometer + Zipkin	Phase 7 — distributed tracing
Metrics	Prometheus + Grafana	Phase 7 — observability dashboard
Docs	Springdoc OpenAPI	Phase 7 — Swagger on all services
DB Migrations	Flyway	All services with a database
Containers	Docker + Docker Compose	Phase 7 — run full system locally
Build tool	Gradle	All services
API Testing	Bruno	All phases — test every endpoint

Final advice

Never rush a phase
If you cannot explain every decision in a phase to a stranger, you are not ready for the next one. The whole point is to deeply understand what you built.

Commit as you go
Every phase should be a separate Git branch and PR into main. Your commit history tells the story of how the system evolved. Interviewers look at this.

Write as you build
Keep a DECISIONS.md file where you document why you made each architectural choice. Update it as you learn. This becomes your interview cheat sheet.

Quantify everything
When you put this on your resume: 'Processed X payments/second', 'p99 latency under Xms', 'achieved 99.9% uptime in testing'. Numbers get you shortlisted.

Depth over breadth
Phases 1–3 done exceptionally well is more impressive than all 7 done shallowly. If time runs out, finish fewer phases and know them cold.

Remember: You started today not knowing what UUID was. You ended today with a working distributed payments API with Redis caching, idempotency, PostgreSQL indexes, and proper transaction handling. In 4–6 months of building this project at 1–2 hours a day, you will be a genuinely strong backend engineer. Trust the process.